

Operators

Lesson link

https://www.w3schools.com/java/java_operators.asp

Operators allow you to manipulate, compare, and perform calculations using primitive data types, followed closely by **control flow statements**. Understanding these concepts allows you to manipulate data and control the order in which your code executes.

Operators	Explanation																					
Arithmetic Operators These perform standard mathematical calculations.	+ (Addition): Adds values together (e.g., 5 + 3 equals 8). - (Subtraction): Subtracts one value from another (e.g., 10 - 4 equals 6). * (Multiplication): Multiplies values (e.g., 3 * 4 equals 12). / (Division): Divides values (e.g., 15 / 3 equals 5). % (Modulus): Returns the remainder of a division (e.g., 7 % 3 equals 1, because 3 goes into 7 twice with 1 left over).																					
Unary Operators Unary operators are the types that need only one operand to perform any operation like increment, decrement, negation, etc. It consists of various arithmetic, logical and other operators that operate on a single operand. Let's look at the various unary operators in detail and see how they operate.	<table><tr><th>Operators</th><th>Description</th></tr><tr><td>+</td><td>Unary plus operator; indicates positive value (numbers are positive without this, however).</td></tr><tr><td>-</td><td>Unary minus operator; negates an expression.</td></tr><tr><td>++</td><td>Increment operator; increments a value by 1.</td></tr><tr><td>--</td><td>Decrement operator; decrements a value by 1.</td></tr><tr><td>!</td><td>Logical complement operator; inverts the value of a boolean.</td></tr></table>	Operators	Description	+	Unary plus operator; indicates positive value (numbers are positive without this, however).	-	Unary minus operator; negates an expression.	++	Increment operator; increments a value by 1.	--	Decrement operator; decrements a value by 1.	!	Logical complement operator; inverts the value of a boolean.									
Operators	Description																					
+	Unary plus operator; indicates positive value (numbers are positive without this, however).																					
-	Unary minus operator; negates an expression.																					
++	Increment operator; increments a value by 1.																					
--	Decrement operator; decrements a value by 1.																					
!	Logical complement operator; inverts the value of a boolean.																					
Assignment Operators These store values inside variables	<table><tr><th>Operator</th><th>Operation</th><th>Equivalent to</th></tr><tr><td>=</td><td>num = 5</td><td>num = 5</td></tr><tr><td>+=</td><td>num+=5</td><td>num = num+5</td></tr><tr><td>-=</td><td>num-=5</td><td>num = num-5</td></tr><tr><td>*=</td><td>num*=5</td><td>num = num*5</td></tr><tr><td>/=</td><td>num/=5</td><td>num = num/5</td></tr><tr><td>%=</td><td>num%=5</td><td>num = num%5</td></tr></table>	Operator	Operation	Equivalent to	=	num = 5	num = 5	+=	num+=5	num = num+5	-=	num-=5	num = num-5	*=	num*=5	num = num*5	/=	num/=5	num = num/5	%=	num%=5	num = num%5
Operator	Operation	Equivalent to																				
=	num = 5	num = 5																				
+=	num+=5	num = num+5																				
-=	num-=5	num = num-5																				
=	num=5	num = num*5																				
/=	num/=5	num = num/5																				
%=	num%=5	num = num%5																				
Ternary Operator Conditional operator that takes three operands. It's a one-liner replacement for the if-then-else statement	variable = Expression1 ? Expression2 : Expression3																					

Comparison / Relational Operators

They return a boolean result after the comparison and are extensively used in looping statements as well as conditional if-else statements and so on.

Operators	Operations
==	Equal to
!=	Not Equal to
>	Greater than
>=	Greater than equal to
<	Less than
<=	Less than equal to

These compare two values and always result in a boolean (true or false).

- == (Equal to): Checks if two values are the same (e.g., 5 == 5 is true).
- != (Not equal to): Checks if two values are different (e.g., 5 != 3 is true).
- > and < (Greater / Less than): Compares sizes (e.g., 10 > 20 is false).
- >= and <= (Greater / Less than or equal to): Includes the boundary number (e.g., 5 >= 5 is true).

Logical Operators

These combine multiple comparison conditions together to make complex decisions.

One thing to keep in mind is, while using AND operator, the second condition is not evaluated if the first one is false. Whereas while using OR operator, the second condition is not evaluated if the first one is true

Logical Operators in Java

1. Logical AND Operator (& and &&)

Operand 1	Operand 2	Returned Value
False	False	False
False	True	False
True	False	False
True	True	True

2. Logical OR Operator (| and ||)

Operand 1	Operand 2	Returned Value
False	False	False
False	True	True
True	False	True
True	True	True

3. Logical NOT Operator (!)

Operand	Returned Value
False	True
True	False

Operator precedence

Operator precedence determines the order in which different operators are evaluated in a single expression. Think of it as the programming version of **PEMDAS** (Parentheses, Exponents, Multiplication, Division, Addition, Subtraction) from math class.

Without precedence rules, a computer wouldn't know how to solve a mixed expression like $5 + 3 * 2$.

The Core Rules of Precedence

When an expression contains multiple operators, Java evaluates them from highest precedence to lowest precedence.

Here is a simplified table of common Java operators, ranked from **highest priority to lowest**:

Precedence	Operator Type	Symbols	Description
1 (Highest)	Parentheses	<code>()</code>	Forces grouping and overrides everything else.
2	Postfix / Unary	<code>++</code> , <code>--</code> , <code>!</code>	Increments, decrements, and logical NOT.
3	Multiplicative	<code>*</code> , <code>/</code> , <code>%</code>	Multiplication, division, and modulus.
4	Additive	<code>+</code> , <code>-</code>	Addition and subtraction.
5	Relational	<code><</code> , <code>></code> , <code><=</code> , <code>>=</code>	Comparisons.
6	Equality	<code>==</code> , <code>!=</code>	Checking if values match.
7	Logical AND	<code>&&</code>	Evaluates conditions together.
8	Logical OR	<code> </code>	Evaluates optional conditions.
9 (Lowest)	Assignment	<code>=</code> , <code>+=</code> , <code>-=</code>	Storing the final result.

Associativity

When two operators have the **exact same** precedence level, Java uses **associativity** to decide the direction of execution.

- **Left-to-Right:** Most operators (like +, -, *, /) execute from left to right.
- **Right-to-Left:** Assignment operators (like =) execute from right to left.

Arithmetic Mix int result = 10 + 5 * 2;	• Step 1: Multiplication (*) has higher precedence than addition (+). Java calculates 5 * 2 first, which is 10. • Step 2: The expression becomes 10 + 10. • Step 3: Java adds them together to get 20
Using Parentheses to Override int result = (10 + 5) * 2;	• Step 1: Parentheses () have the highest absolute precedence. Java calculates 10 + 5 first, which is 15. • Step 2: The expression becomes 15 * 2. • Step 3: Java multiplies them to get 30.
Mixed Logic and Comparison boolean check = 5 + 3 > 10 && 2 == 2;	• Step 1 (Arithmetic): 5 + 3 is evaluated first. The expression is now 8 > 10 && 2 == 2. • Step 2 (Relational): 8 > 10 is evaluated. It is false. The expression is now false && 2 == 2. • Step 3 (Equality): 2 == 2 is evaluated. It is true. The expression is now false && true. • Step 4 (Logical AND): false && true evaluates to false